

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего
образования



НИЖЕГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ им. Р.Е.АЛЕКСЕЕВА

Институт радиоэлектроники и информационных технологий
Кафедра информатики и систем управления

Лабораторная работа №2 Тема: «Создание веб-сайтов»
(наименование темы проекта или работы)

ОТЧЕТ

по лабораторной работе

(курсовому проекту, ОТЧЕТ по лабораторной работе)
по дисциплине

Информатика и компьютерные технологии
(наименование дисциплины)

РУКОВОДИТЕЛЬ:

(подпись)

Савкин А.Е.
(фамилия, и.,о.)

СТУДЕНТ:

(подпись)

Ташмирзаев А.А.
(фамилия, и.,о.)

25-ПОз
(шифр группы)

Работа защищена «__» _____

С оценкой _____

Нижний Новгород 2026

Задание к выполнению лабораторной работы № 2

Текст задания:

Необходимо разработать веб-сайт с использованием HTML, CSS и JavaScript. Сайт должен содержать несколько вкладок, элементы интерфейса и интерактивные функции. В отчёте необходимо представить задание, цель работы, описание хода выполнения со скриншотами, вывод и приложение с текстом программы.

Цель работы:

Изучить основы создания веб-сайтов и на практике применить HTML, CSS и JavaScript при разработке собственного проекта.

Ход работы:

Перед началом выполнения лабораторной работы была выбрана идея проекта. В качестве темы был разработан сайт-тренажёр скорости печати. Основная задача сайта - позволить пользователю тренироваться в наборе текста и получать статистику о скорости печати, количестве ошибок и точности.

Сайт реализован как одностраничное веб-приложение с несколькими вкладками. Переключение между разделами осуществляется без перезагрузки страницы. В верхней части страницы расположено меню навигации, с помощью которого пользователь может переходить между основными разделами сайта.

Основные разделы сайта:

- Главная страница
- Тренировка
- Свой текст
- О сайте

На главной странице размещено описание проекта и кнопка перехода к тренировке. Основной раздел сайта - страница тренировки, где пользователю предлагается текст для набора. Во время печати программа отслеживает ввод символов и сравнивает их с исходным текстом.

Если пользователь вводит символ правильно, прогресс увеличивается. При ошибке фиксируется неправильный символ и пользователю необходимо его исправить. После завершения текста выводится результат тренировки.

В разделе «Свой текст» пользователь может вставить любой текст и проверить скорость печати на собственном материале.

HTML используется для построения структуры страниц сайта, CSS отвечает за оформление элементов интерфейса, а JavaScript реализует основную интерактивную логику: обработку ввода текста, подсчёт ошибок, расчёт скорости печати и отображение статистики.

Для разработки сайта использовался редактор Visual Studio Code.

Тестирование работы сайта проводилось в браузере Яндекс.

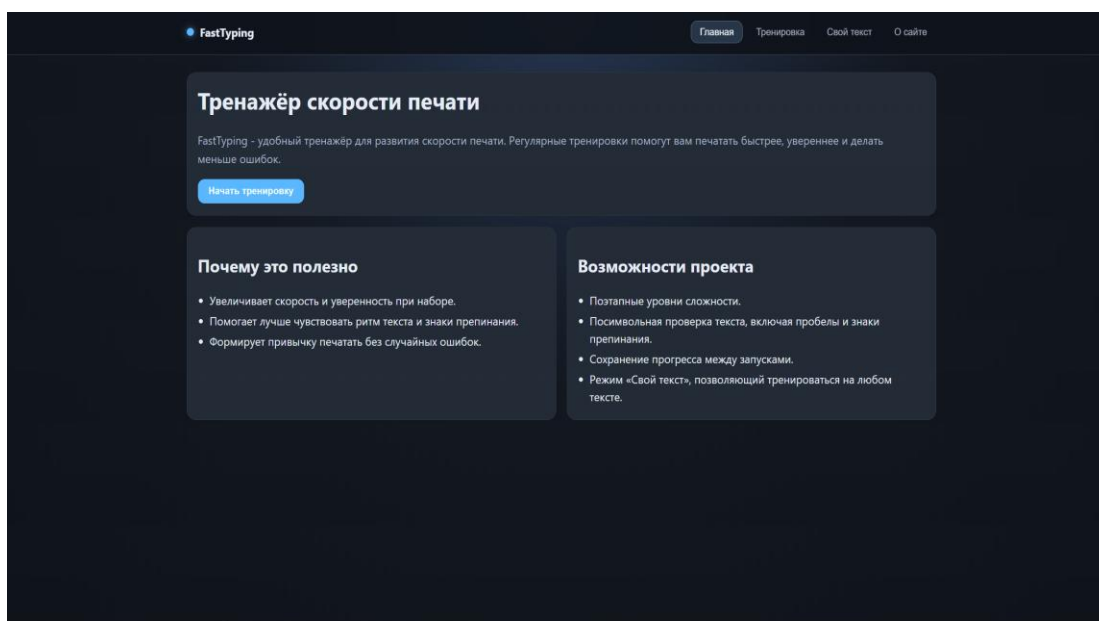


Рисунок 1 - Главная страница сайта FastTyping

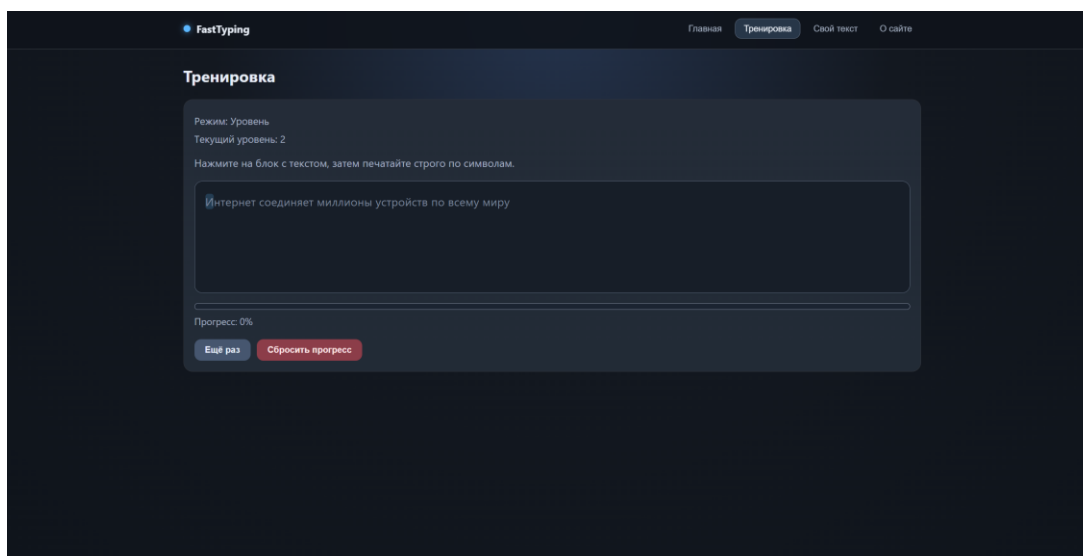


Рисунок 2 - Страница тренировки печати

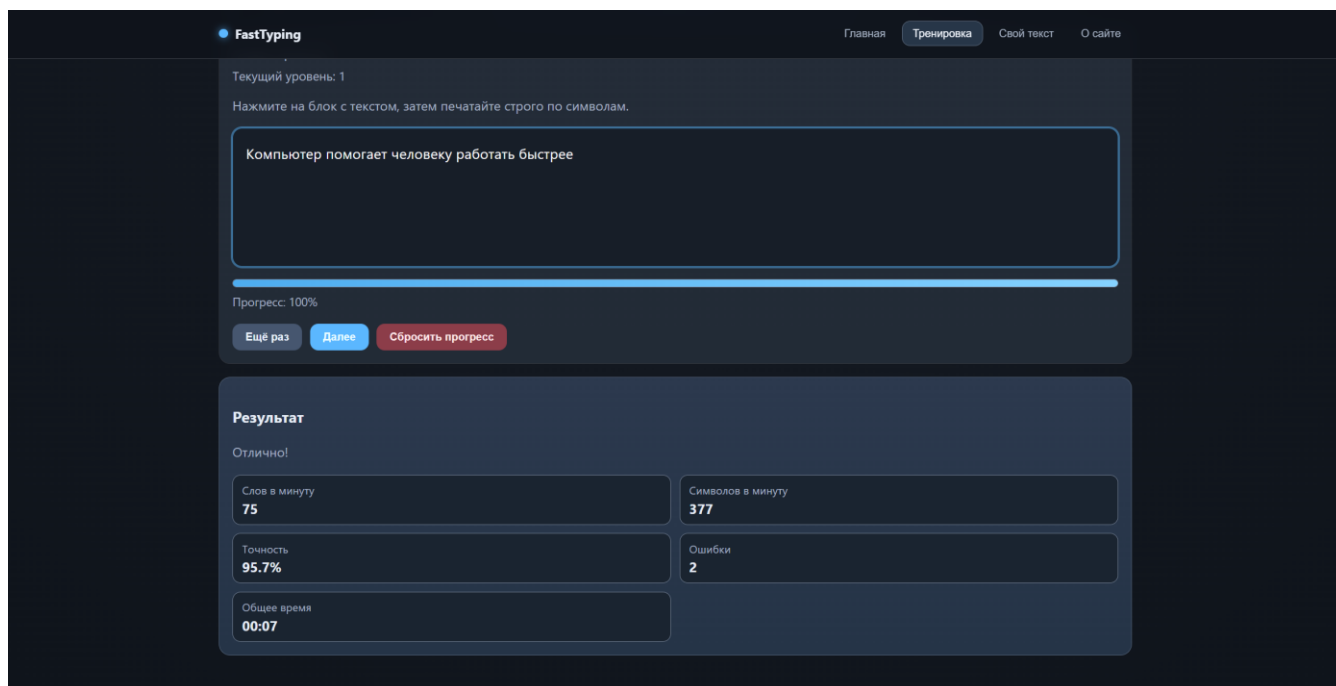


Рисунок 3 - Результаты после завершения тренировки

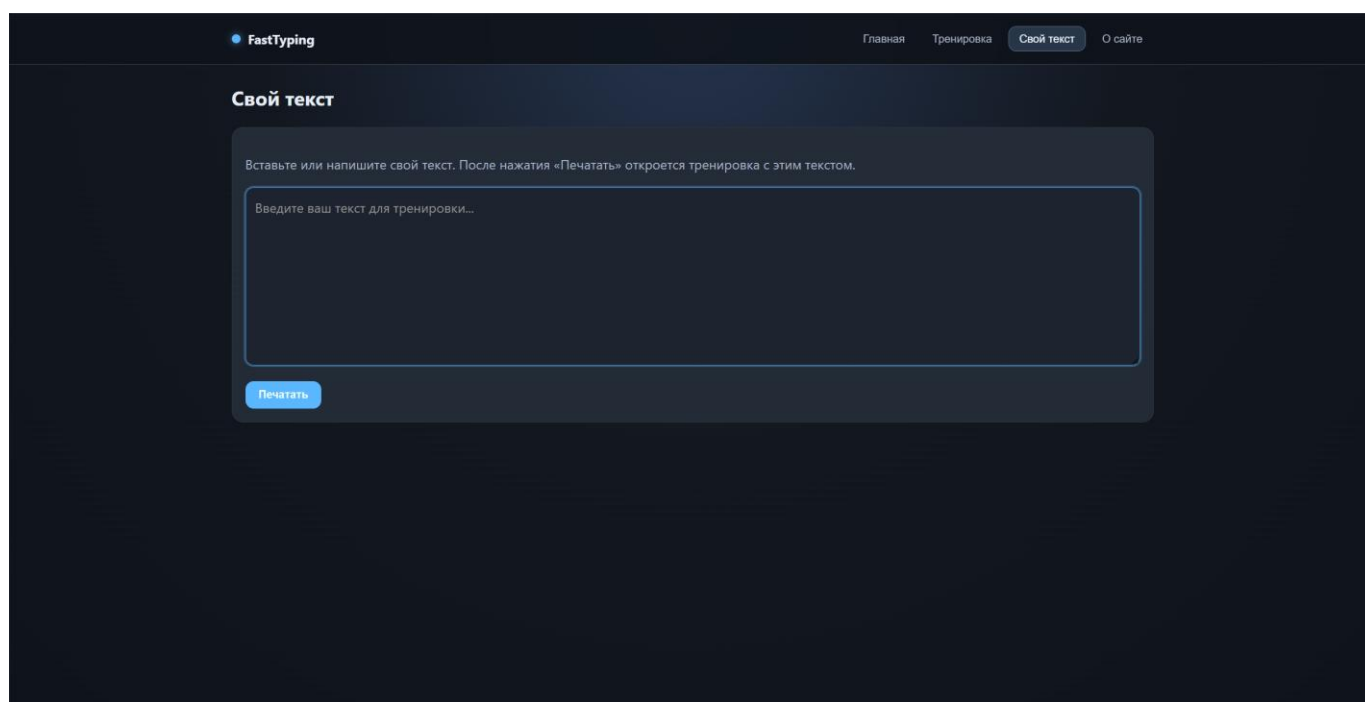


Рисунок 4 - Режим «Свой текст»

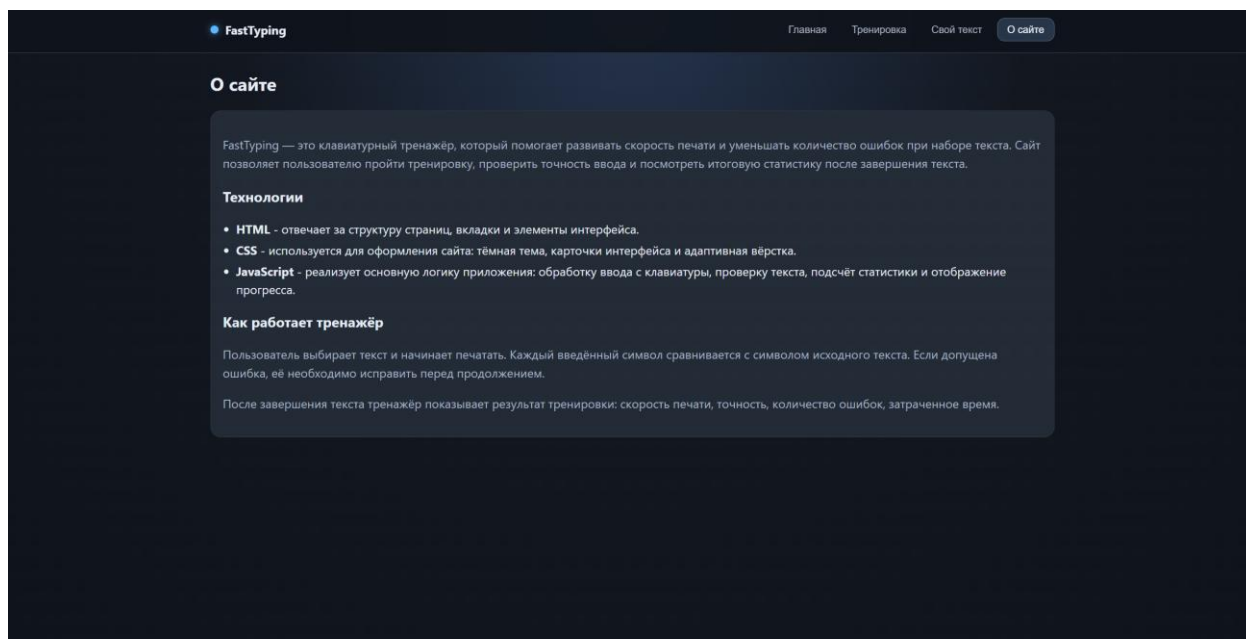


Рисунок 5 – «О сайте»

Вывод:

В ходе выполнения лабораторной работы был разработан сайт-тренажёр скорости печати FastTyping.

При создании сайта были использованы технологии HTML, CSS и JavaScript.

В результате был получен рабочий сайт, позволяющий тренировать скорость печати, отслеживать ошибки и получать итоговую статистику.

Таким образом, цель лабораторной работы была достигнута.

Приложение

Код файла index.html:

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Тренажёр скорости печати</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header class="site-header">
    <div class="container header-row">
      <div class="brand">
        <span class="brand-dot"></span>
        <span class="brand-name">FastTyping</span>
      </div>

      <nav class="main-nav" aria-label="Навигация">
        <button class="nav-link active" data-section="home">Главная</button>
        <button class="nav-link" data-section="training">Тренировка</button>
        <button class="nav-link" data-section="custom">Свой текст</button>
        <button class="nav-link" data-section="about">О сайте</button>
      </nav>
    </div>
  </header>

  <main class="container main-content">
    <section id="home" class="page-section active">
      <div class="card hero-card">
        <h1>Тренажёр скорости печати</h1>
        <p>
```

```

FastTyping - удобный тренажёр для развития скорости печати.
Регулярные тренировки помогут вам печатать быстрее, увереннее и делать меньше ошибок.
</p>
<button id="goToTrainingBtn" class="btn btn-primary">Начать тренировку</button>
</div>

<div class="grid two-columns">
  <article class="card">
    <h2>Почему это полезно</h2>
    <ul class="clean-list">
      <li>Увеличивает скорость и уверенность при наборе.</li>
      <li>Помогает лучше чувствовать ритм текста и знаки препинания.</li>
      <li>Формирует привычку печатать без случайных ошибок.</li>
    </ul>
  </article>

  <article class="card">
    <h2>Возможности проекта</h2>
    <ul class="clean-list">
      <li>Поэтапные уровни сложности.</li>
      <li>Посимвольная проверка текста, включая пробелы и знаки препинания.</li>
      <li>Сохранение прогресса между запусками.</li>
      <li>Режим «Свой текст», позволяющий тренироваться на любом тексте.</li>
    </ul>
  </article>
</div>
</section>

<section id="training" class="page-section">
  <h2 class="section-title">Тренировка</h2>

  <div class="card training-card">
    <p class="meta-line">Режим: <span id="modelLabel">Уровень</span></p>
    <p class="meta-line">Текущий уровень: <span id="levelLabel">1</span></p>
    <p class="instruction" id="trainingInstruction">
      Нажмите на блок с текстом, затем печатайте строго по символам.
    </p>

    <div
      id="typingArea"
      class="typing-area blurred"
      tabindex="0"
      role="button"
      aria-label="Область тренировки печати"
    >
      <div id="startOverlay" class="start-overlay">Нажмите, чтобы начать</div>
      <div id="textDisplay" class="text-display" aria-live="polite"></div>
    </div>

    <div class="progress-box" aria-label="Прогресс прохождения текста">
      <div class="progress-track">
        <div id="progressBar" class="progress-bar"></div>
      </div>
      <p id="progressText" class="progress-text">Прогресс: 0%</p>
    </div>

    <div class="btn-row">
      <button id="retryBtn" class="btn btn-secondary">Ещё раз</button>
      <button id="nextLevelBtn" class="btn btn-primary hidden">Далее</button>
      <button id="resetProgressBtn" class="btn btn-danger">Сбросить прогресс</button>
    </div>
  </div>

  <div id="resultCard" class="card result-card hidden" aria-live="polite">
    <h3>Результат</h3>
    <p id="resultMessage" class="result-message"></p>

    <div class="result-grid">
      <div><span>Слов в минуту</span><strong id="finalWpm">0</strong></div>
      <div><span>Символов в минуту</span><strong id="finalCpm">0</strong></div>
      <div><span>Точность</span><strong id="finalAccuracy">0%</strong></div>
      <div><span>Ошибки</span><strong id="finalErrors">0</strong></div>
      <div><span>Общее время</span><strong id="finalTime">00:00</strong></div>
    </div>
  </div>
</section>

<section id="custom" class="page-section">
  <h2 class="section-title">Свой текст</h2>

  <div class="card">
    <p>Вставьте или напишите свой текст. После нажатия «Печатать» откроется тренировка с этим текстом.</p>
    <textarea
      id="customTextInput"
      rows="8"

```

```

        placeholder="Введите ваш текст для тренировки..."
        spellcheck="false"
    ></textarea>

    <div class="btn-row">
        <button id="useCustomTextBtn" class="btn btn-primary">Печатать</button>
    </div>

    <p id="customWarning" class="warning hidden">Поле пустое. Добавьте текст перед запуском.</p>
</div>
</section>

<section id="about" class="page-section">
    <h2 class="section-title">О сайте</h2>

    <div class="card">
        <p>
            FastTyping – это клавиатурный тренажёр, который помогает развивать скорость печати и уменьшать количество ошибок при наборе текста.
            Сайт позволяет пользователю пройти тренировку, проверить точность ввода и посмотреть итоговую статистику после завершения текста.
        </p>

        <h3>Технологии</h3>
        <ul class="clean-list">
            <li><strong>HTML</strong> - отвечает за структуру страниц, вкладки и элементы интерфейса.</li>
            <li><strong>CSS</strong> - используется для оформления сайта: тёмная тема, карточки интерфейса и адаптивная вёрстка.</li>
            <li><strong>JavaScript</strong> - реализует основную логику приложения: обработку ввода с клавиатуры, проверку текста, подсчёт статистики и отображение прогресса.</li>
        </ul>

        <h3>Как работает тренажёр</h3>
        <p>
            Пользователь выбирает текст и начинает печатать.
            Каждый введённый символ сравнивается с символом исходного текста.
            Если допущена ошибка, её необходимо исправить перед продолжением.
        </p>
        <p>
            После завершения текста тренажёр показывает результат тренировки: скорость печати, точность, количество ошибок, затраченное время.
        </p>
    </div>
</section>
</main>

<script src="levels.js"></script>
<script src="script.js"></script>
</body>
</html>

```

Листинг 1 - Файл index.html

Код файла style.css:

```

:root {
    --bg: #131820;
    --bg-soft: #1d2430;
    --card: #242c38;
    --card-border: #364255;
    --text-main: #e8eef7;
    --text-muted: #a7b3c6;
    --accent: #5bb7ff;
    --accent-strong: #3f9fe7;
    --danger: #ff6b6b;
    --success: #98e08e;
    --pending: #778297;
}

* {
    box-sizing: border-box;
}

body {
    margin: 0;
    min-height: 100vh;
    color: var(--text-main);
    font-family: "Segoe UI", Tahoma, Geneva, Verdana, sans-serif;
    background: radial-gradient(circle at top, #25344e 0%, var(--bg) 44%, #0f141b 100%);
}

```

Нижний Новгород 2026

```

.container {
  width: min(1120px, 92%);
  margin: 0 auto;
}

.site-header {
  position: sticky;
  top: 0;
  z-index: 25;
  background: rgba(14, 19, 27, 0.9);
  border-bottom: 1px solid #2b374a;
  backdrop-filter: blur(8px);
}

.header-row {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 1rem;
  padding: 0.95rem 0;
}

.brand {
  display: flex;
  align-items: center;
  gap: 0.55rem;
  font-weight: 700;
}

.brand-dot {
  width: 11px;
  height: 11px;
  border-radius: 999px;
  background: var(--accent);
  box-shadow: 0 0 10px rgba(91, 183, 255, 0.7);
}

.main-nav {
  display: flex;
  flex-wrap: wrap;
  gap: 0.4rem;
}

.nav-link {
  border: 1px solid transparent;
  border-radius: 10px;
  background: transparent;
  color: var(--text-muted);
  padding: 0.45rem 0.8rem;
  cursor: pointer;
  transition: all 0.2s ease;
}

.nav-link:hover {
  color: var(--text-main);
  border-color: #3d4c63;
}

.nav-link.active {
  color: #f4f8ff;
  border-color: #415873;
  background: #273648;
}

.main-content {
  padding: 1.5rem 0 2.4rem;
}

.page-section {
  display: none;
  animation: fadeIn 0.25s ease;
}

.page-section.active {
  display: block;
}

@keyframes fadeIn {
  from {
    opacity: 0;
    transform: translateY(5px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}

```



```

}
}

.card {
  background: linear-gradient(180deg, var(--card) 0%, #222b36 100%);
  border: 1px solid var(--card-border);
  border-radius: 14px;
  padding: 1rem;
  margin-bottom: 1rem;
}

.hero-card h1,
.section-title {
  margin-top: 0;
}

.hero-card p,
.card p {
  color: var(--text-muted);
  line-height: 1.55;
}

.grid {
  display: grid;
  gap: 1rem;
}

.two-columns {
  grid-template-columns: repeat(2, minmax(0, 1fr));
}

.clean-list {
  margin: 0;
  padding-left: 1.1rem;
  line-height: 1.5;
}

.clean-list li {
  margin-bottom: 0.25rem;
}

.meta-line {
  margin: 0.25rem 0;
  font-size: 0.95rem;
}

.instruction {
  margin: 0.75rem 0 0.95rem;
}

.typing-area {
  position: relative;
  min-height: 170px;
  border: 1px solid #4b5f7c;
  border-radius: 12px;
  background: #171e28;
  padding: 1rem;
  cursor: text;
  user-select: none;
}

.typing-area:focus-visible {
  outline: 2px solid rgba(91, 183, 255, 0.45);
  border-color: var(--accent);
}

.typing-area.blurred .text-display {
  filter: blur(2.2px);
  opacity: 0.82;
}

.start-overlay {
  position: absolute;
  inset: 0;
  display: grid;
  place-items: center;
  background: rgba(16, 22, 30, 0.45);
  color: #eef6ff;
  font-weight: 600;
  border-radius: 12px;
  text-align: center;
  pointer-events: none;
}

.text-display {

```

```

white-space: pre-wrap;
word-break: break-word;
font-size: 1.08rem;
line-height: 1.7;
letter-spacing: 0.01em;
}

.char {
  transition: color 0.15s ease, background 0.15s ease;
  border-radius: 4px;
}

.char.correct {
  color: #ffffff;
}

.char.error {
  color: var(--danger);
}

.char.pending {
  color: var(--pending);
}

.char.current {
  background: rgba(91, 183, 255, 0.2);
}

.progress-box {
  margin-top: 0.95rem;
}

.progress-track {
  height: 10px;
  border-radius: 999px;
  background: #1b2431;
  border: 1px solid #314059;
  overflow: hidden;
}

.progress-bar {
  height: 100%;
  width: 0;
  background: linear-gradient(90deg, #4dacef 0%, #85d2ff 100%);
  transition: width 0.18s ease;
}

.progress-text {
  margin: 0.45rem 0 0;
  font-size: 0.9rem;
  color: var(--text-muted);
}

.btn-row {
  display: flex;
  flex-wrap: wrap;
  gap: 0.6rem;
  margin-top: 0.95rem;
}

.btn {
  border: 0;
  border-radius: 10px;
  padding: 0.56rem 1rem;
  font-weight: 600;
  cursor: pointer;
  transition: transform 0.15s ease, opacity 0.15s ease, background 0.15s ease;
}

.btn:hover {
  transform: translateY(-1px);
}

.btn:active {
  transform: translateY(0);
}

.btn-primary {
  background: var(--accent);
  color: #0f1f31;
}

.btn-primary:hover {
  background: var(--accent-strong);
}

```

```

.btn-secondary {
  background: #46566f;
  color: #edf4ff;
}

.btn-secondary:hover {
  background: #516582;
}

.btn-danger {
  background: #8c3d49;
  color: #ffe9eb;
}

.btn-danger:hover {
  background: #a04755;
}

.result-card {
  border-color: #4a6587;
  background: linear-gradient(180deg, #2c394e 0%, #253447 100%);
}

.result-message {
  margin-top: 0;
  color: #f0f6ff;
}

.result-grid {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 0.65rem;
}

.result-grid div {
  background: #1a2430;
  border: 1px solid #344861;
  border-radius: 10px;
  padding: 0.55rem 0.7rem;
}

.result-grid span {
  display: block;
  color: var(--text-muted);
  font-size: 0.84rem;
  margin-bottom: 0.15rem;
}

.result-grid strong {
  font-size: 1.06rem;
}

textarea {
  width: 100%;
  border: 1px solid #3a4a62;
  border-radius: 10px;
  background: var(--bg-soft);
  color: var(--text-main);
  font: inherit;
  line-height: 1.5;
  padding: 0.7rem;
  resize: vertical;
}

textarea:focus {
  outline: 2px solid rgba(91, 183, 255, 0.35);
  border-color: var(--accent);
}

.warning {
  color: #ff9c9c;
  margin: 0.7rem 0 0;
}

.hidden {
  display: none;
}

@media (max-width: 920px) {
  .two-columns,
  .result-grid {
    grid-template-columns: 1fr;
  }

  .header-row {

```

```

    flex-direction: column;
    align-items: flex-start;
  }
}

```

Листинг 2 - Файл style.css

Код файла script.js:

```

const STORAGE_KEY = 'fastTypingUnlockedLevel';

const state = {
  mode: 'level',
  unlockedLevel: 0,
  currentLevel: 0,
  targetText: '',
  position: 0,
  hasErrorAtCurrent: false,
  attempts: 0,
  mistakes: 0,
  isArmed: false,
  started: false,
  finished: false,
  startTime: 0,
  endTime: 0
};

const navLinks = document.querySelectorAll('.nav-link');
const sections = document.querySelectorAll('.page-section');
const goToTrainingBtn = document.getElementById('goToTrainingBtn');

const modelLabel = document.getElementById('modelLabel');
const levelLabel = document.getElementById('levelLabel');
const trainingInstruction = document.getElementById('trainingInstruction');
const typingArea = document.getElementById('typingArea');
const startOverlay = document.getElementById('startOverlay');
const textDisplay = document.getElementById('textDisplay');

const progressBar = document.getElementById('progressBar');
const progressText = document.getElementById('progressText');

const retryBtn = document.getElementById('retryBtn');
const nextLevelBtn = document.getElementById('nextLevelBtn');
const resetProgressBtn = document.getElementById('resetProgressBtn');

const resultCard = document.getElementById('resultCard');
const resultMessage = document.getElementById('resultMessage');
const finalWpm = document.getElementById('finalWpm');
const finalCpm = document.getElementById('finalCpm');
const finalAccuracy = document.getElementById('finalAccuracy');
const finalErrors = document.getElementById('finalErrors');
const finalTime = document.getElementById('finalTime');

const customTextInput = document.getElementById('customTextInput');
const useCustomTextBtn = document.getElementById('useCustomTextBtn');
const customWarning = document.getElementById('customWarning');

// старт
init();

// инициализация
function init() {
  if (!Array.isArray(window.levels) || window.levels.length === 0) {
    textDisplay.textContent = 'Не удалось загрузить уровни. Проверьте levels.js.';
    return;
  }

  state.unlockedLevel = loadUnlockedLevel();
  state.currentLevel = state.unlockedLevel;

  bindEvents();
  openLevel(state.currentLevel);
  switchSection('home');
}

// события
function bindEvents() {
  navLinks.forEach((link) => {
    link.addEventListener('click', () => {
      switchSection(link.dataset.section);
    });
  });
}

```

Нижний Новгород 2026

```

    });

    goToTrainingBtn.addEventListener('click', () => {
        if (state.mode !== 'level') {
            openLevel(state.unlockedLevel);
        }
        switchSection('training');
    });

    typingArea.addEventListener('click', armTraining);
    document.addEventListener('keydown', handleKeydown);

    retryBtn.addEventListener('click', restartCurrentSession);

    nextLevelBtn.addEventListener('click', () => {
        if (state.currentLevel < window.levels.length - 1) {
            openLevel(state.currentLevel + 1);
        }
    });

    resetProgressBtn.addEventListener('click', resetProgress);
    useCustomTextBtn.addEventListener('click', startCustomTraining);
}

// переключение вкладок
function switchSection(sectionId) {
    sections.forEach((section) => {
        section.classList.toggle('active', section.id === sectionId);
    });

    navLinks.forEach((link) => {
        link.classList.toggle('active', link.dataset.section === sectionId);
    });
}

// запуск уровня
function openLevel(levelIndex) {
    state.mode = 'level';
    state.currentLevel = clampLevel(levelIndex);
    state.targetText = String(window.levels[state.currentLevel] || '').replace(/\r/g, '');

    modeLabel.textContent = 'Уровень';
    levelLabel.textContent = String(state.currentLevel + 1);
    trainingInstruction.textContent =
        'Нажмите на блок с текстом, затем печатайте строго по символам.';

    resetSessionState();
}

// запуск своего текста
function startCustomTraining() {
    const userText = customTextInput.value.replace(/\r/g, '');

    if (!userText.trim()) {
        customWarning.classList.remove('hidden');
        return;
    }

    customWarning.classList.add('hidden');

    state.mode = 'custom';
    state.targetText = userText;

    modeLabel.textContent = 'Свой текст';
    levelLabel.textContent = '-';
    trainingInstruction.textContent =
        'Режим своего текста. Ошибки исправляйте Backspace.';

    resetSessionState();
    switchSection('training');
}

// сброс текущей сессии
function resetSessionState() {
    state.position = 0;
    state.hasErrorAtCurrent = false;
    state.attempts = 0;
    state.mistakes = 0;
    state.isArmed = false;
    state.started = false;
    state.finished = false;
    state.startTime = 0;
    state.endTime = 0;

    typingArea.classList.add('blurred');

```

```

startOverlay.classList.remove('hidden');

resultCard.classList.add('hidden');
nextLevelBtn.classList.add('hidden');

renderText();
updateProgress();
}

// активация тренировки
function armTraining() {
  if (state.finished || !state.targetText) {
    return;
  }

  state.isArmed = true;
  typingArea.classList.remove('blurred');
  startOverlay.classList.add('hidden');
  typingArea.focus();
}

// обработка ввода
function handleKeydown(event) {
  const isTrainingOpen = document.getElementById('training').classList.contains('active');

  if (!isTrainingOpen || !state.isArmed || state.finished) {
    return;
  }

  if (event.ctrlKey || event.metaKey || event.altKey) {
    return;
  }

  if (event.key === 'Backspace') {
    event.preventDefault();
    handleBackspace();
    return;
  }

  const typedChar = normalizeKey(event);
  if (typedChar === null) {
    return;
  }

  event.preventDefault();

  if (!state.started) {
    state.started = true;
    state.startTime = Date.now();
  }

  const expectedChar = state.targetText[state.position];
  state.attempts += 1;

  if (typedChar === expectedChar) {
    state.position += 1;
    state.hasErrorAtCurrent = false;

    if (state.position === state.targetText.length) {
      finishSession();
      return;
    }
  } else {
    state.hasErrorAtCurrent = true;
    state.mistakes += 1;
  }

  renderText();
  updateProgress();
}

// исправление ошибки
function handleBackspace() {
  if (!state.started || state.finished) {
    return;
  }

  if (state.hasErrorAtCurrent) {
    state.hasErrorAtCurrent = false;
  } else if (state.position > 0) {
    state.position -= 1;
  }

  renderText();
  updateProgress();
}

```

```

}

// завершение и подсчёт
function finishSession() {
  state.finished = true;
  state.endTime = Date.now();

  renderText();
  updateProgress();

  const elapsedSeconds = Math.max(1, Math.round((state.endTime - state.startTime) / 1000));
  const minutes = elapsedSeconds / 60;
  const correctChars = state.targetText.length;

  const wpm = Math.round((correctChars / 5) / minutes);
  const cpm = Math.round(correctChars / minutes);
  const accuracy = state.attempts > 0
    ? ((state.attempts - state.mistakes) / state.attempts) * 100
    : 100;

  showResult({
    wpm,
    cpm,
    accuracy,
    errors: state.mistakes,
    elapsedSeconds
  });

  if (state.mode === 'level') {
    unlockNextLevel();

    if (state.currentLevel < window.levels.length - 1) {
      nextLevelBtn.classList.remove('hidden');
    }
  }
}

// вывод результата
function showResult(stats) {
  finalWpm.textContent = String(stats.wpm);
  finalCpm.textContent = String(stats.cpm);
  finalAccuracy.textContent = `${stats.accuracy.toFixed(1)}%`;
  finalErrors.textContent = String(stats.errors);
  finalTime.textContent = formatTime(stats.elapsedSeconds);

  if (stats.accuracy >= 95) {
    resultMessage.textContent = 'Отлично!';
  } else if (stats.accuracy >= 85) {
    resultMessage.textContent = 'Хороший результат.';
  } else {
    resultMessage.textContent = 'Попробуйте ещё раз, чтобы улучшить скорость.';
  }

  resultCard.classList.remove('hidden');
}

// кнопка "ещё раз"
function restartCurrentSession() {
  if (state.mode === 'level') {
    openLevel(state.currentLevel);
  } else {
    modeLabel.textContent = 'Свой текст';
    levelLabel.textContent = '-';
    resetSessionState();
  }
}

// сброс прогресса
function resetProgress() {
  const shouldReset = window.confirm('Сбросить прогресс уровней?');
  if (!shouldReset) {
    return;
  }

  state.unlockedLevel = 0;
  localStorage.setItem(STORAGE_KEY, '0');

  if (state.mode === 'level') {
    openLevel(0);
  }
}

// открытие следующего уровня
function unlockNextLevel() {
  const nextLevel = state.currentLevel + 1;

```

```

    if (nextLevel >= window.levels.length) {
      return;
    }

    if (nextLevel > state.unlockedLevel) {
      state.unlockedLevel = nextLevel;
      localStorage.setItem(STORAGE_KEY, String(state.unlockedLevel));
    }
  }

  // отрисовка текста
  function renderText() {
    const fragment = document.createDocumentFragment();

    for (let i = 0; i < state.targetText.length; i += 1) {
      const span = document.createElement('span');
      span.classList.add('char');
      span.textContent = state.targetText[i];

      if (i < state.position) {
        span.classList.add('correct');
      } else if (i === state.position && !state.finished) {
        span.classList.add(state.hasErrorAtCurrent ? 'error' : 'pending');
        span.classList.add('current');
      } else {
        span.classList.add('pending');
      }

      fragment.appendChild(span);
    }

    textDisplay.innerHTML = '';
    textDisplay.appendChild(fragment);
  }

  // прогресс-бар
  function updateProgress() {
    const progress = state.targetText.length > 0
      ? Math.round((state.position / state.targetText.length) * 100)
      : 0;

    progressBar.style.width = `${progress}%`;
    progressText.textContent = `Прогресс: ${progress}%`;
  }

  // преобразование клавиши
  function normalizeKey(event) {
    if (event.key === 'Enter') {
      return '\n';
    }

    if (event.key === 'Tab') {
      return '\t';
    }

    if (event.key.length === 1) {
      return event.key;
    }

    return null;
  }

  // загрузка прогресса
  function loadUnlockedLevel() {
    const raw = Number(localStorage.getItem(STORAGE_KEY));

    if (!Number.isInteger(raw) || raw < 0) {
      return 0;
    }

    return Math.min(raw, window.levels.length - 1);
  }

  // ограничение номера уровня
  function clampLevel(levelIndex) {
    if (!Number.isInteger(levelIndex)) {
      return 0;
    }

    if (levelIndex < 0) {
      return 0;
    }

    if (levelIndex > state.unlockedLevel) {

```



```

    return state.unlockedLevel;
  }

  return Math.min(levelIndex, window.levels.length - 1);
}

// формат времени
function formatTime(totalSeconds) {
  const minutes = Math.floor(totalSeconds / 60).toString().padStart(2, '0');
  const seconds = Math.floor(totalSeconds % 60).toString().padStart(2, '0');
  return `${minutes}:${seconds}`;
}

```

Листинг 3 - Файл script.js

Код файла levels.js:

```

// Набор текстов для 10 уровней: от простого к сложному.
window.levels = [
  "Компьютер помогает человеку работать быстрее",
  "Интернет соединяет миллионы устройств по всему миру",
  "Программисты создают сайты, приложения и различные цифровые сервисы",
  "HTML используется для создания структуры веб-страницы",
  "CSS отвечает за внешний вид сайта: цвета, шрифты и расположение элементов",
  "JavaScript добавляет интерактивность на сайте - кнопки, анимации и реакции на действия пользователя",
  "Современные веб-сайты обычно строятся на основе трех технологий: HTML, CSS и JavaScript",
  "Когда пользователь открывает страницу, браузер загружает HTML, применяет стили CSS и выполняет JavaScript",
  "Фронтенд-разработчик отвечает за внешний интерфейс сайта, а бэкенд-разработчик работает с сервером и базой данных",
  "Финальный уровень: веб-разработка объединяет множество технологий - HTML, CSS, JavaScript, серверы, базы данных и API, которые вместе создают современные интерактивные сайты"
];

```

Листинг 4 – Файл levels.js